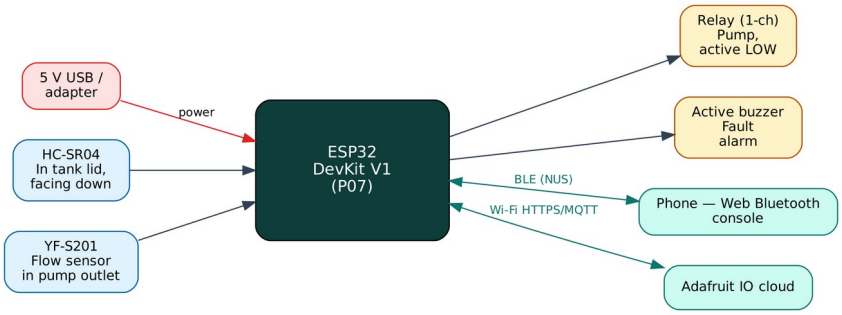
A PROJECT REPORT ON

Water Tank Level Monitor and Smart Pump Controller

IoT project using ESP32 with Bluetooth Low Energy and Wi-Fi (Adafruit IO)



Project: P07 · Advanced level

Peripherals: HC-SR04 ultrasonic sensor, YF-S201 flow sensor, Relay module, Active buzzer

Submitted by: _____

Under the guidance of: _____

Department / Institution: _____

October 2026

Certificate

This is to certify that the project report entitled **"Water Tank Level Monitor and Smart Pump Controller"** is a bonafide record of the work carried out by _____ (Roll No. _____) in partial fulfilment of the requirements for the award of _____ in _____ during the academic year _____.

Project Guide: _____ Head of Department: _____

Date: _____ Place: _____

Acknowledgement

I express my sincere gratitude to my project guide _____ for continuous guidance and support, to the Head of the Department _____, and to all faculty members and friends who helped me complete this project successfully.

Contents

Abstract **6**

1. Introduction **7**

1.1 Problem Statement 7
1.2 Objectives 7
1.3 Scope of the Project 7
1.4 Organisation of the Report 7

2. Existing System and Proposed System **8**

2.1 Existing Systems 8
2.2 Proposed System 8
2.3 Why Both BLE and Wi-Fi? 8

3. System Design **9**

3.1 Block Diagram 9
3.2 System Architecture 9
3.3 Hardware Components and Cost 9
3.4 ESP32 DevKit V1 10
3.5 Working Principle of Components 10

4. Circuit Design **11**

4.1 Circuit Diagram 11
4.2 Pin Connections 11
4.3 Interfacing and Power Notes 12

5. Data Flow Diagrams **13**

5.1 DFD Level 0 (Context Diagram) 13
5.2 DFD Level 1 14
5.3 Data Dictionary 14

6. Software Design and Implementation **16**

6.1 Development Environment 16
6.2 Firmware Architecture 16
6.3 Working of the System 16
6.4 Flowchart 17
6.5 BLE Communication Protocol 17
6.6 Cloud Integration (Adafruit IO) 18
6.7 Configurable Settings 19

7. Setup and Deployment Guide **20**

7.1 Hardware Assembly 20
7.2 Flashing the Firmware 20
7.3 Cloud Setup (Adafruit IO) 20
7.4 BLE App Setup (Web Bluetooth Console) 21
7.5 First-Run Checklist 22

8. Testing and Results **23**

8.1 Expected Output 23
8.2 Troubleshooting 23

9. Discussion **24**

9.1 Advantages 24
9.2 Limitations 24
9.3 Safety Precautions 24
9.4 Security Considerations 24
9.5 Future Scope 24

10. Conclusion..... 25

References..... 26

Appendix A: Project Source Code..... 27

Appendix B: Shared Library Code..... 31

Abstract

Overhead water tanks in Indian homes either overflow because the motor is left on or run dry because nobody notices. This project measures the water level with an ultrasonic sensor in the tank lid and the actual water flow with a YF-S201 Hall-effect flow sensor, and switches the pump through a relay. In AUTO mode the pump starts below a low threshold and stops at a high threshold. Three protections stop the pump: dry-run (pump on but no flow for 30 s), overflow (level above 98 %) and maximum run time. Level, flow and pump state are published to Adafruit IO, where the dashboard can also switch the pump and change the mode. A phone connected over BLE shows a live tank gauge and allows calibration of the tank dimensions.

Keywords: ESP32, Internet of Things, Bluetooth Low Energy, Wi-Fi, Adafruit IO, HC-SR04 ultrasonic sensor, YF-S201 flow sensor, Relay module, Active buzzer.

1. Introduction

The Internet of Things (IoT) connects physical devices — sensors and actuators — to networks so that data can be collected, analysed and acted on automatically. The ESP32 from Espressif Systems has made IoT affordable: a single ₹350 board combines a dual-core 240 MHz processor, Wi-Fi and Bluetooth Low Energy. This project uses both radios at once: BLE for direct, local interaction with a phone and Wi-Fi for cloud connectivity through Adafruit IO, a free IoT/cloud platform.

1.1 Problem Statement

Manual pump control wastes water and electricity through overflow and damages pumps through dry running when the supply line is empty. Float switches are mechanical, wear out and give no remote information.

1.2 Objectives

1. Measure tank level without contact and show it as a percentage.
2. Control the pump automatically with hysteresis between low and high levels.
3. Detect dry running using a flow sensor and stop the pump.
4. Prevent overflow and limit continuous run time.
5. Monitor and control remotely (Adafruit IO) and locally (BLE).

1.3 Scope of the Project

Domestic overhead tanks up to about 3 m deep with single-phase pumps up to 1 HP switched through a suitable contactor. The demo uses a 5 V mini pump and a bucket.

1.4 Organisation of the Report

Chapter 2 reviews existing approaches and describes the proposed system. Chapter 3 covers system design and components, Chapter 4 the circuit, Chapter 5 the data flow diagrams, Chapter 6 the software, Chapter 7 the step-by-step setup of hardware, cloud and app, Chapter 8 testing, Chapter 9 discussion and Chapter 10 the conclusion. Complete source code is given in the appendices.

2. Existing System and Proposed System

2.1 Existing Systems

Typical water-level controllers use probes or float switches and only switch the pump on level; they do not detect dry running and offer no remote monitoring. Many ultrasonic projects use single readings that jump because of ripples.

2.2 Proposed System

The proposed system improves on these in the following ways:

- Median-of-5 filtering of ultrasonic readings.
- Flow-based dry-run protection and maximum run-time protection.
- Fault latch: the pump cannot restart until the fault is cleared by the user.
- Pump never switches on from a stale cloud value after a reboot.

2.3 Why Both BLE and Wi-Fi?

Wi-Fi gives the device internet access for remote monitoring, history and control from anywhere, but it depends on a router and an internet connection, and the device needs the Wi-Fi password before it can connect. BLE needs no infrastructure: any modern phone can connect directly within about 10-30 m. Using both gives:

- **Provisioning** — Wi-Fi credentials and cloud keys are sent from the phone over BLE and stored in flash; nothing secret is hard-coded or uploaded to GitHub.
- **Resilience** — local monitoring and control keep working when the internet or router is down.
- **Remote access** — the cloud dashboard works from anywhere when the internet is available.
- **Low cost** — no extra hardware; both radios are built into the ESP32.

The ESP32 shares one 2.4 GHz radio between Wi-Fi and Bluetooth by time-division coexistence. For this to work the Wi-Fi modem-sleep mode must stay enabled, which the firmware ensures. NimBLE, a lightweight BLE stack, is used instead of the default Bluedroid stack to leave more RAM for TLS (HTTPS/MQTTS) connections.

3. System Design

3.1 Block Diagram

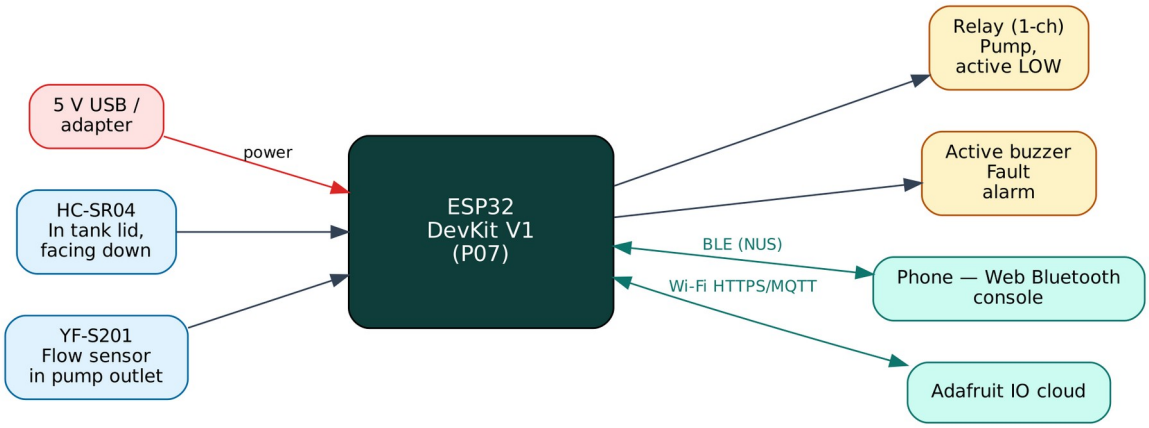


Figure 1: Block diagram of the Water Tank & Pump Controller

The ESP32 is the central controller. Input devices (blue) feed measurements to it, output devices (amber) are driven by it, and two communication links (green) connect it to the user's phone over BLE and to the Adafruit IO cloud over Wi-Fi. The board is powered from 5 V through its USB connector or VIN pin.

3.2 System Architecture

The system follows the standard three-layer IoT architecture:

- **Perception layer** — HC-SR04 ultrasonic sensor, YF-S201 flow sensor, Relay module, Active buzzer connected to the ESP32 GPIO, ADC, I2C, SPI or UART.
- **Network layer** — BLE 4.2 GATT server (Nordic UART Service) and Wi-Fi 802.11 b/g/n with TLS-encrypted MQTT to Adafruit IO.
- **Application layer** — the Web Bluetooth console on the phone and the Adafruit IO dashboard / database.

3.3 Hardware Components and Cost

#	Component	Qty	Specification	Cost
1	ESP32 DevKit V1 (ESP-WROOM-32, 30-pin)	1	Dual-core 240 MHz, Wi-Fi + BLE, 4 MB flash	₹350
2	Breadboard 830-point + jumper wires (M-M, M-F)	1	Prototyping and connections	₹150
3	Micro-USB data cable	1	Power + programming	₹60
4	HC-SR04 ultrasonic sensor (or JSN-SR04T waterproof)	1	2-400 cm	₹60
5	YF-S201 water flow sensor	1	1-30 L/min, ½-inch, Hall-effect	₹180
6	1-channel 5 V relay module (10 A)	1	Pump switching	₹50
7	Active buzzer 5 V	1	Fault alarm	₹15
8	5 V mini submersible pump + 1 m tube	1	Demo load	₹90

#	Component	Qty	Specification	Cost
	(demo)			
9	Resistors 1 kΩ/2 kΩ and 10 kΩ/20 kΩ	1	Level dividers	₹5
	Total (approx.)			**₹960**

Table 1: Bill of materials with approximate Indian prices (Robu.in / Amazon.in / local market, 2026)

> Prices are indicative and vary by seller. The breadboard, cable and ESP32 can be reused across all ten projects, so the incremental cost of this project is about ₹400.

3.4 ESP32 DevKit V1

Feature	Value
Module	ESP-WROOM-32 (ESP32-D0WD), dual-core Xtensa LX6 up to 240 MHz
Memory	520 KB SRAM, 4 MB SPI flash
Wireless	Wi-Fi 802.11 b/g/n 2.4 GHz; Bluetooth 4.2 BR/EDR + BLE
I/O	25 usable GPIO on the 30-pin board, 12-bit ADC (ADC1: GPIO32-39), 2 × DAC, PWM on any output
Interfaces	3 × UART, 2 × I2C, 3 × SPI, I2S, touch, RMT
Supply	5 V via USB/VIN, on-board 3.3 V regulator (AMS1117)
Logic level	3.3 V — inputs are NOT 5 V tolerant

Table 2: Key specifications of the ESP32 DevKit V1

3.5 Working Principle of Components

Ultrasonic level

The sensor measures the distance d from the lid to the water surface. With D = distance to the tank bottom and F = distance to the full line, level % = (D − d) / (D − F) × 100.

YF-S201

Water turns a small rotor with a magnet; a Hall-effect sensor gives one pulse per magnet pass. The datasheet relation is f (Hz) = 7.5 × Q (L/min), i.e. about 450 pulses per litre. Pulses are counted in an interrupt service routine.

Relay

Opto-isolated relay switches the pump (or the coil of a contactor for large pumps).

Adafruit IO

MQTT broker and dashboards; feeds hold time-series values and blocks (gauge, toggle) display or change them.

4. Circuit Design

4.1 Circuit Diagram

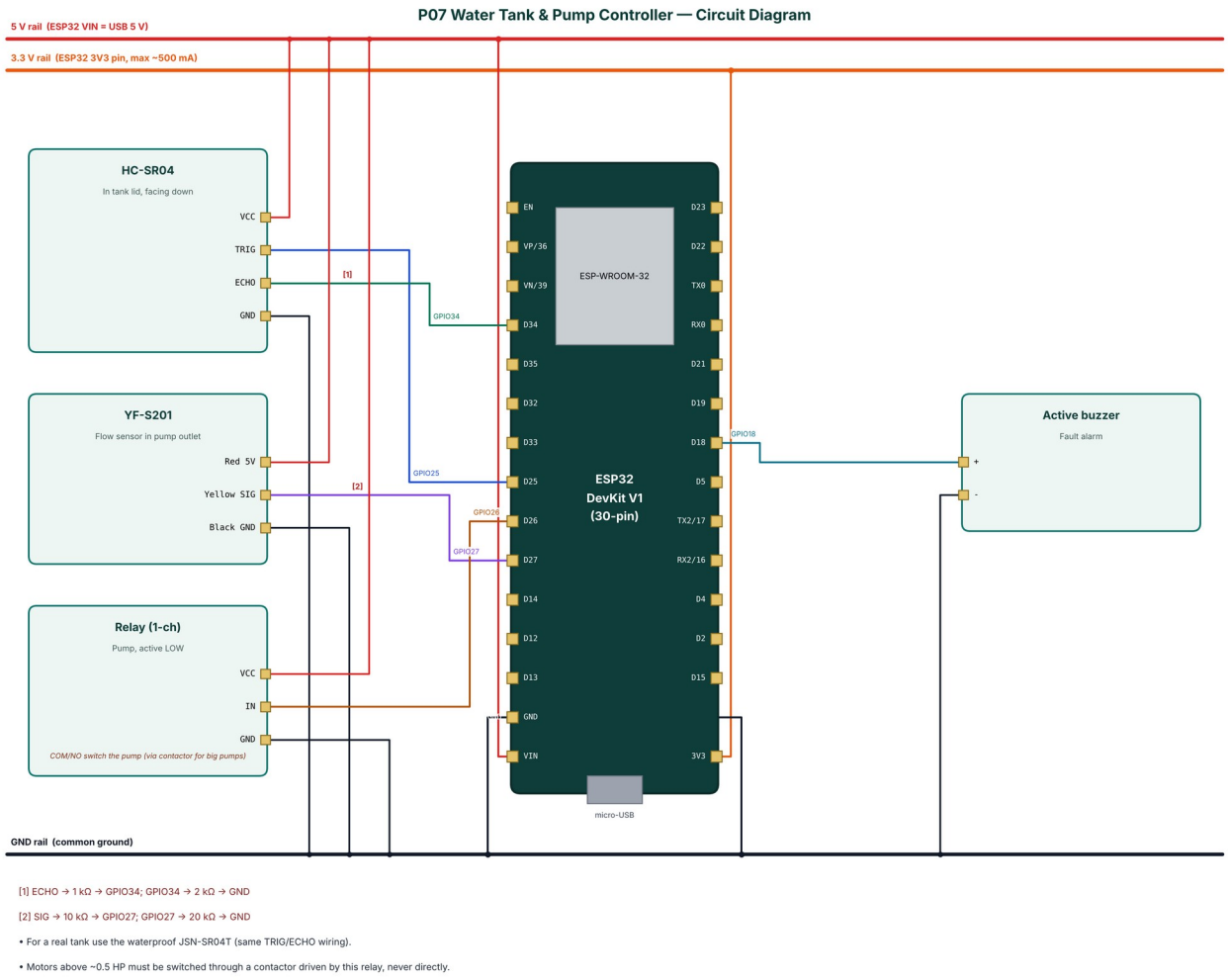


Figure 2: Circuit diagram of the Water Tank & Pump Controller

4.2 Pin Connections

Module	Module pin	ESP32 connection	Notes
HC-SR04	VCC	5V rail (VIN)	
HC-SR04	TRIG	GPIO25	
HC-SR04	ECHO	GPIO34	ECHO -> 1 kΩ -> GPIO34; GPIO34 -> 2 kΩ -> GND
HC-SR04	GND	GND	
YF-S201	Red 5V	5V rail (VIN)	
YF-S201	Yellow SIG	GPIO27	SIG -> 10 kΩ -> GPIO27; GPIO27 -> 20 kΩ -> GND
YF-S201	Black GND	GND	
Relay (1-ch)	VCC	5V rail (VIN)	
Relay (1-ch)	IN	GPIO26	

Module	Module pin	ESP32 connection	Notes
Relay (1-ch)	GND	GND	
Active buzzer	+	GPIO18	
Active buzzer	-	GND	

Table 3: Pin connection table

4.3 Interfacing and Power Notes

- For a real tank use the waterproof JSN-SR04T (same TRIG/ECHO wiring).
- Motors above ~0.5 HP must be switched through a contactor driven by this relay, never directly.
- Only ADC1 pins (GPIO32-39) are used for analog inputs, because ADC2 cannot be read while Wi-Fi is active.
- Strapping pins GPIO0, 2, 12 and 15 are avoided for inputs so the board always boots normally.
- All grounds are common. 5 V signals are reduced to 3.3 V with resistor dividers before reaching the ESP32.

5. Data Flow Diagrams

Data flow diagrams (DFDs) show how data moves through the system without showing control logic or timing. External entities are drawn as rectangles, processes as numbered circles, data stores as open boxes and data flows as labelled arrows.

5.1 DFD Level 0 (Context Diagram)

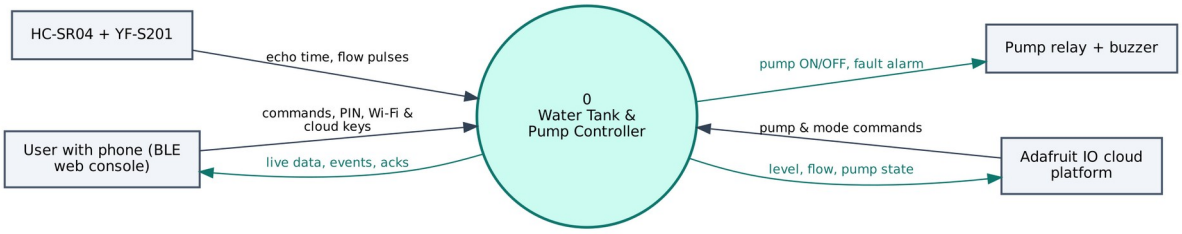


Figure 3: DFD Level 0 — context diagram

At level 0 the whole Water Tank & Pump Controller is a single process. It receives echo time, flow pulses from the HC-SR04 + YF-S201, commands and configuration from the user's phone, and commands from Adafruit IO. It sends pump ON/OFF, fault alarm to the Pump relay + buzzer, live data to the phone and level, flow, pump state to Adafruit IO.

5.2 DFD Level 1

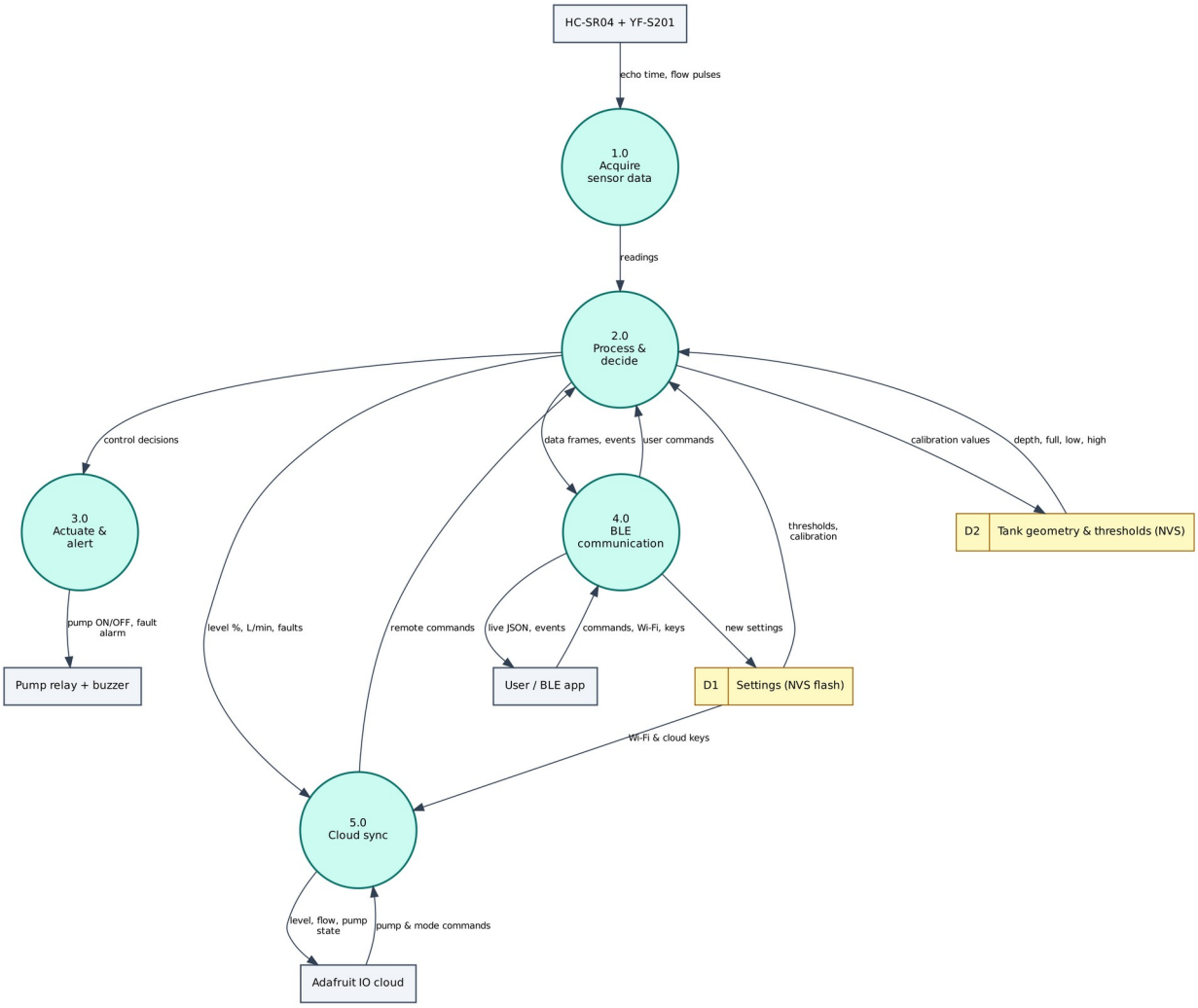


Figure 4: DFD Level 1 — main processes and data stores

Process / store	Description
1.0 Acquire sensor data	Reads echo time, flow pulses from the HC-SR04 + YF-S201; filters/averages the raw values.
2.0 Process & decide	Computes level %, L/min, faults; applies thresholds and rules; generates events.
3.0 Actuate & alert	Drives the Pump relay + buzzer (pump ON/OFF, fault alarm).
4.0 BLE communication	Sends JSON frames and events to the phone; receives and authenticates commands.
5.0 Cloud sync	Sends level, flow, pump state to Adafruit IO; receives pump & mode commands.
D1 Settings (NVS)	Wi-Fi credentials, cloud keys, PIN, thresholds — survive reboots.
D2 Tank geometry & thresholds (NVS)	Stores calibration values; returns depth, full, low, high.

Table 4: Description of level-1 processes and data stores

5.3 Data Dictionary

Field	Meaning	Unit
lvl	Tank level	%

Field	Meaning	Unit
cm	Measured distance	cm
flow	Flow rate	L/min
lit	Litres pumped	L
pump	Pump ON	bool
mode	AUTO / MANUAL	—
fault	Active fault text	—
cloud	MQTT connected	bool

Table 5: BLE data frame fields

Cloud object	Content / usage
p07-level	Gauge 0-100 %
p07-flow	Line chart, L/min
p07-pump	Toggle block ON/OFF (remote pump switch)
p07-mode	Toggle block AUTO/MANUAL

Table 6: Adafruit IO data objects

6. Software Design and Implementation

6.1 Development Environment

Item	Details
IDE / build	Arduino IDE 2.x or arduino-cli; GitHub Actions builds firmware in the cloud
Board package	esp32 by Espressif Systems 3.x (tested with 3.3.12)
Board settings	ESP32 Dev Module, Partition Scheme: Minimal SPIFFS (1.9MB APP with OTA/190KB SPIFFS)
Common libraries	NimBLE-Arduino 2.x, ArduinoJson 7.x, PubSubClient 2.8
Project libraries	none beyond the common ones
Phone app	Web Bluetooth console (Chrome on Android / desktop, Bluefy on iPhone); nRF Connect as alternative
Cloud	Adafruit IO (free tier)

Table 7: Software tools

> The default 1.2 MB app partition is too small for BLE + Wi-Fi + TLS. Select **Minimal SPIFFS (1.9MB APP)** in Tools → Partition Scheme, otherwise the upload fails with 'Sketch too big'.

6.2 Firmware Architecture

The firmware is split into three layers so that the ten projects share tested code:

- **IoTCore.h** (shared) — Wi-Fi station with auto-reconnect, NimBLE GATT server with the Nordic UART Service, line-based command parser shared by BLE and USB serial, PIN authentication, settings in NVS (Preferences), NTP time in IST, JSON helpers.
- **Cloud.h** (shared) — ThingSpeak HTTPS client, Adafruit IO MQTT/TLS client with a 28 messages/minute rate limiter, Supabase PostgREST client (insert, update, select).
- **P07_Water_Tank_Pump_Controller.ino** — sensor drivers, algorithms, automation rules and the project's own commands.

The main loop is non-blocking: every task is scheduled with millis() timers, BLE writes are queued from the BLE callback into a FreeRTOS queue and executed in loop(), so the BLE stack never waits for slow operations such as HTTPS requests.

6.3 Working of the System

Every second the firmware takes five ultrasonic readings 15 ms apart and uses the median, converts it to a level percentage using the stored tank geometry, and computes flow rate from the pulses counted in the last second. Total litres pumped are accumulated.

AUTO mode switches the pump ON when level < 'low' (25 %) and OFF when level ≥ 'high' (90 %). While the pump is ON, flow must exceed 0.5 L/min within 30 s, otherwise a 'Dry run' fault is raised. Level ≥ 98 % raises an 'Overflow risk' fault and run time > 'maxrun' minutes raises 'Max run time reached'. A fault turns the pump off, sounds the buzzer and blocks restarts until 'clear' is sent.

Level and flow are published every 15 s; the pump feed is updated on every change. The device subscribes to p07-pump and p07-mode but does not request their last value after reconnecting, so a stale ON can never start the pump unexpectedly.

6.4 Flowchart

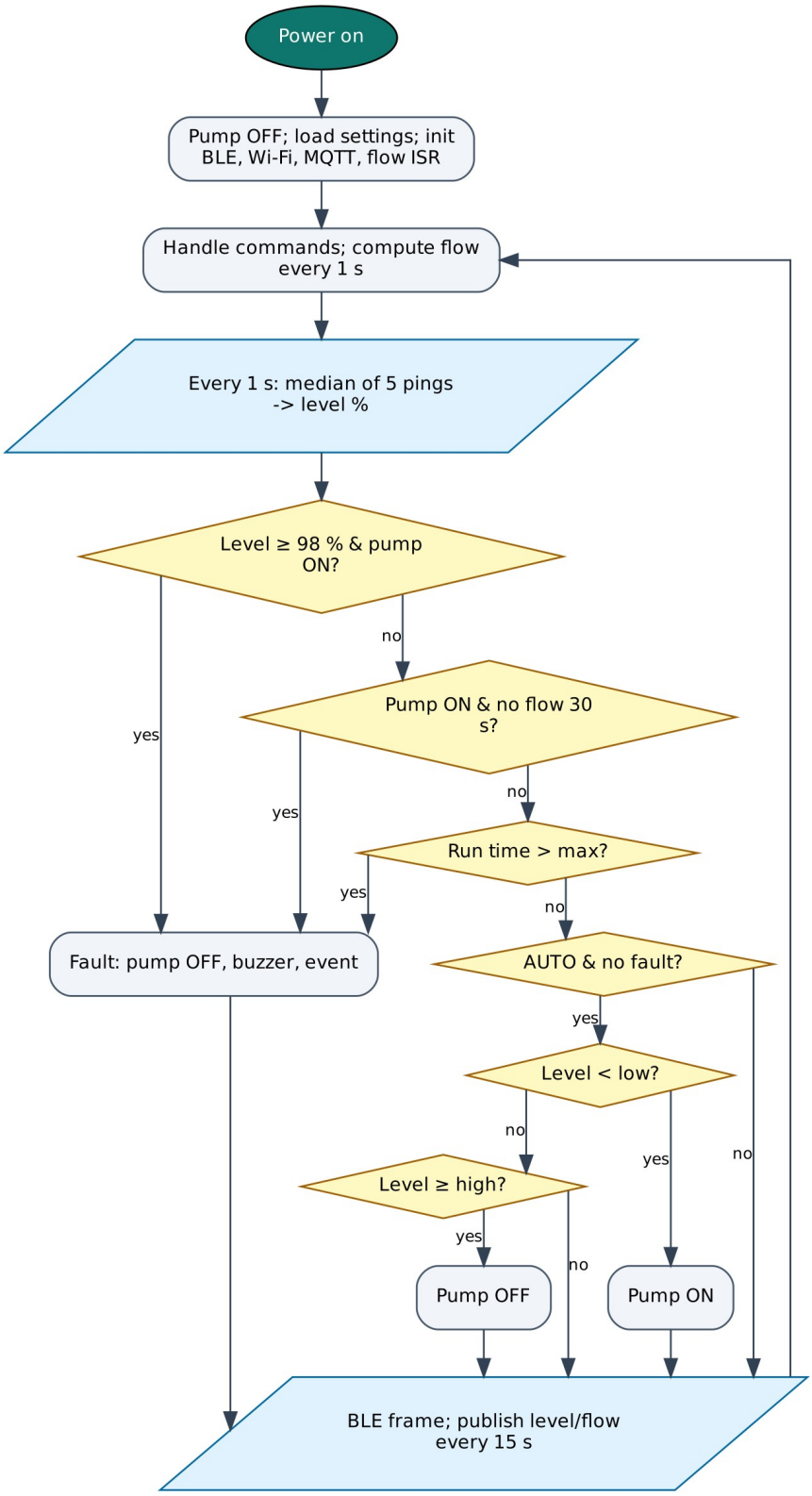


Figure 5: Firmware flowchart

6.5 BLE Communication Protocol

The device is a BLE peripheral named **IOT-P07-XXXX** (XXXX = last bytes of the MAC address). It exposes the Nordic UART Service, which most BLE apps understand:

Item	UUID	Properties
Service (NUS)	6E400001-B5A3-F393-E0A9-E50E24DCCA9E	Primary
RX — phone → ESP32	6E400002-B5A3-F393-E0A9-E50E24DCCA9E	Write, Write without response
TX — ESP32 → phone	6E400003-B5A3-F393-E0A9-E50E24DCCA9E	Notify, Read

Table 8: GATT service and characteristics

Commands are plain text lines (`command:argument`). Replies are one JSON object per line ending in a newline; long lines are split into MTU-sized notifications and re-joined by the app. Commands that change anything require `auth:<PIN>` first on each connection (default PIN 1234, change it with `set:pin=XXXX`).

Command	PIN needed	Action
auth:<PIN>	—	Unlock for this connection
status	No	Status JSON: Wi-Fi, IP, RSSI, heap, uptime, time
wifi:<SSID>,<password>	Yes	Save Wi-Fi credentials and connect
set:<key>=<value>	Yes	Save a setting (cloud keys, thresholds)
get:<key>	Yes	Read a setting (secrets masked)
reboot / forget	Yes	Restart / erase all settings
pump:on / pump:off	Yes	Manual pump control
mode:auto / mode:manual	Yes	Select mode
clear	Yes	Clear fault and silence buzzer
resetvol	Yes	Reset litres counter

Table 9: BLE commands (common commands first, then project commands)

Example data frame sent by the device:

```
{"lvl":64,"cm":45.2,"flow":3.12,"lit":12.4,"pump":true,"mode":"AUTO","fault":"","cloud":true,"type":"dat
a"}
```

6.6 Cloud Integration (Adafruit IO)

The device keeps a persistent MQTT connection to io.adafruit.com on port 8883 (TLS) using the account username and key. Values are published to `username/feeds/<feed-key>`, and control feeds are subscribed so dashboard changes arrive within about a second. A built-in limiter keeps the device below Adafruit IO's free limit of 30 data points per minute.

Object	Content
p07-level	Gauge 0-100 %
p07-flow	Line chart, L/min
p07-pump	Toggle block ON/OFF (remote pump switch)
p07-mode	Toggle block AUTO/MANUAL

Table 10: Adafruit IO configuration

6.7 Configurable Settings

Key	Default	Meaning
aiouser / aiokey	—	Adafruit IO credentials
depth	100	Sensor to tank bottom (cm)
fullcm	15	Sensor to full line (cm)
low / high	25 / 90	AUTO thresholds (%)
maxrun	30	Maximum continuous run (min)
ssid / pass	—	Wi-Fi network (set with the wifi: command)
pin	1234	BLE PIN

Table 11: Settings stored in flash (change with set:key=value)

7. Setup and Deployment Guide

7.1 Hardware Assembly

1. Place the ESP32 DevKit across the centre gap of the breadboard so both pin rows are free.
2. Connect the ESP32 3V3 pin to one breadboard power rail (3.3 V), VIN to the other rail (5 V) and both GND pins to the ground rails.
3. Wire each module exactly as in the pin connection table (section 4.2), including the resistor dividers marked with [n] in the circuit diagram.
4. Check every connection against the circuit diagram before powering on. Make sure no 5 V signal reaches an ESP32 pin directly.
5. Connect the USB cable to a computer (for flashing) or a 5 V adapter (for normal use).

7.2 Flashing the Firmware

Option A — Arduino IDE

1. Install Arduino IDE 2.x from arduino.cc.
2. File → Preferences → Additional boards manager URLs: `https://espressif.github.io/arduino-esp32/package_esp32_index.json`.
3. Tools → Board → Boards Manager: install **esp32 by Espressif Systems** (version 3.x).
4. Sketch → Include Library → Manage Libraries: install **NimBLE-Arduino** (2.x), **ArduinoJson** (7.x), **PubSubClient**.
5. Download the repository and open the folder **P07_Water_Tank_Pump_Controller** (it already contains IoTCore.h, Cloud.h and config.h).
6. Tools: Board **ESP32 Dev Module**, Partition Scheme **Minimal SPIFFS (1.9MB APP with OTA/190KB SPIFFS)**, Upload Speed 921600, and the correct COM port.
7. Click Upload. If it stays at 'Connecting...', hold the BOOT button on the ESP32 until upload starts.
8. Open Serial Monitor at 115200 baud and type `status` to check the device.

Option B — no software installation (GitHub Actions + browser flasher)

1. Open the repository https://github.com/rohitkukreja705/esp32-iot-projects → **Actions** → latest **Build firmware** run.
2. Download the artifact **firmware-P07_Water_Tank_Pump_Controller** and unzip it to get `P07_Water_Tank_Pump_Controller.ino.merged.bin`.
3. Open **https://espressif.github.io/esptool-js/** in Chrome or Edge on a computer, click **Connect** and select the ESP32's serial port.
4. Set Flash Address **0x0**, choose the merged .bin file and click **Program**. Press the EN (reset) button when done.

7.3 Cloud Setup (Adafruit IO)

1. Go to **io.adafruit.com** and create a free account.
2. Click the yellow **key icon (My Key / Adafruit IO Key)** and copy your **Username** and **Active Key**.

3. In the BLE web console open **Setup**, unlock and save them in `aiouser` and `aiokey` (commands: `set:aiouser=NAME`, `set:aiokey=aio_xxxx`).
4. Power the device: feeds are created automatically on the first publish. You can also create them first in **Feeds → New Feed** using the exact keys in the table below.
5. Open **Dashboards → New Dashboard**, then **+ (Create New Block)**. Use Toggle blocks for switch feeds with button-on text `ON` and button-off text `OFF` (or `AUTO`/`MANUAL` for mode feeds), Gauge blocks for levels and Line Chart blocks for history.
6. Free tier: 30 data points per minute, 10 feeds, 5 dashboards and 30 days of history per account. Projects on the same account share these limits, so delete feeds of projects you are not running or use separate accounts.

Field / feed / table	Content
p07-level	Gauge 0-100 %
p07-flow	Line chart, L/min
p07-pump	Toggle block ON/OFF (remote pump switch)
p07-mode	Toggle block AUTO/MANUAL

Table 12: Items to create in Adafruit IO

7.4 BLE App Setup (Web Bluetooth Console)

No app installation is needed. The console is a web page hosted free on GitHub Pages; Chrome talks to the ESP32 directly through the Web Bluetooth API.

1. On an Android phone turn on **Bluetooth** and **Location** (Android requires location for BLE scanning).
2. Open **https://rohitkukreja705.github.io/esp32-iot-projects/?p=P07** in **Chrome**. (On iPhone use the free **Bluefy** browser; on a laptop use Chrome or Edge.)
3. Tap **Connect** and choose **IOT-P07-XXXX** in the pop-up list. The dot turns green and the Live tab starts updating.
4. Open **Setup**, enter the PIN (default **1234**) and tap **Unlock**.
5. Enter your 2.4 GHz Wi-Fi name and password and tap **Save Wi-Fi**. After a few seconds the status line shows the IP address.
6. Enter the Adafruit IO keys from section 7.3 and tap **Save** next to each.
7. Use the **Control** tab for commands and the **Log** tab to see every event and acknowledgement.

Alternative: nRF Connect app

1. Install **nRF Connect for Mobile** (Nordic Semiconductor, free) and scan.
2. Connect to **IOT-P07-XXXX** and open **Nordic UART Service**.
3. Enable notifications on the **TX** characteristic (6E400003…).
4. Write UTF-8 text to the **RX** characteristic (6E400002…), e.g. `auth:1234`, then `status`.

> To host the console yourself: in the GitHub repository open Settings → Pages, choose 'Deploy from a branch', branch main, folder /docs. Web Bluetooth only works on HTTPS pages, which GitHub Pages provides.

7.5 First-Run Checklist

- Serial Monitor shows '[BLE] advertising as IOT-P07-XXXX'.
- Status in the console shows wifi: true and an IP address.
- Data appears in Adafruit IO within a minute.
- All sensor values in the Live tab are plausible (no '—').

8. Testing and Results

The following test cases verify each function. Record the observed result and mark Pass/Fail while testing your build.

#	Test	Procedure	Expected result	Observed
T1	Level calibration	Set depth to bucket depth, fullcm to sensor-to-brim minus 2 cm	Empty ≈ 0 %, full ≈ 100 %	
T2	Auto fill	Lower the water below 25 %	Pump starts; stops at 90 %	
T3	Dry run	Lift the pump out of the water while it runs	Pump stops after ~30 s; 'Dry run' fault + buzzer	
T4	Fault latch	Try pump:on while a fault is active	'pump_blocked' event; pump stays OFF until 'clear'	
T5	Remote control	Switch p07-mode to MANUAL and toggle p07-pump	Pump follows dashboard	
T6	Reboot safety	Leave p07-pump ON in the cloud, reboot the ESP32	Pump stays OFF after boot	
T7	Flow accuracy	Pump into a 1 L measuring jug	Litres counter ≈ 1.0 L (±10 %)	

Table 13: Test cases

8.1 Expected Output

A typical data frame received in the console's Log tab looks like this (values will differ):

```
{"lvl":64,"cm":45.2,"flow":3.12,"lit":12.4,"pump":true,"mode":"AUTO","fault":"","cloud":true,"type":"data"}
```

Serial Monitor (115200 baud) prints the same lines along with connection messages such as `[WiFi] connected`, `[BLE] connected` and the HTTP or MQTT result of each cloud upload, which helps in troubleshooting.

8.2 Troubleshooting

Symptom	Likely cause and fix
Device not in the Connect list	Bluetooth/Location off on the phone, or the page is not opened in Chrome over HTTPS.
'Sketch too big' on upload	Select Partition Scheme 'Minimal SPIFFS (1.9MB APP with OTA)'.
Wi-Fi never connects	ESP32 supports only 2.4 GHz networks; check the SSID/password (case-sensitive).
Settings refused	Send auth:<PIN> first; the unlock is per BLE connection.
Sensor shows null / —	Check wiring and supply voltage; see the Serial Monitor for 'not found' messages.
Board resets randomly	Weak USB supply or servo/relay current; use a 5 V 2 A adapter and add a 470 µF capacitor.
No data in Adafruit IO	Wrong username/key, rate limit exceeded or 10-feed limit reached.

Table 14: Troubleshooting guide

9. Discussion

9.1 Advantages

- Prevents overflow and protects the pump from dry running.
- Non-contact level measurement — no corroding probes in the water.
- Remote visibility of tank level and water usage.
- Safe defaults: fault latch and no auto-start from stale cloud data.

9.2 Limitations

- HC-SR04 is not waterproof; condensation inside a tank shortens its life (use JSN-SR04T).
- YF-S201 accuracy is about ±10 % and needs a minimum flow of ~1 L/min.
- Large pumps need a contactor and motor-protection relay.
- Adafruit IO free tier keeps 30 days of history.

9.3 Safety Precautions

- The pump is a mains appliance: use a contactor, fuse and proper enclosure; follow electrical codes.
- Seal the electronics against humidity; mount the controller outside the tank.

9.4 Security Considerations

- Secrets (Wi-Fi password, cloud keys) are stored in the ESP32's NVS flash, not in source code.
- BLE commands that change the device require a PIN; change the default PIN after setup.
- Cloud traffic is TLS-encrypted. Certificate verification is disabled (setInsecure) for simplicity; for production, pin the server's root certificate with setCACert().
- Use a dedicated cloud account/project for the device and rotate keys if the device is lost.

9.5 Future Scope

- Add a sump-tank level sensor so the pump never starts when the sump is empty.
- Daily water-usage report and leak detection (flow with pump off).
- Solar-powered remote tank node using ESP-NOW or LoRa.
- Voltage monitoring to protect the motor from low-voltage starts.

10. Conclusion

The controller automates tank filling while protecting both the tank and the pump, using a flow sensor to detect dry running that level-only controllers miss. Remote monitoring through Adafruit IO and local control over BLE make it convenient, and conservative defaults make it safe to leave unattended.

All design goals listed in Chapter 1 are met with low-cost parts available in the Indian market, free cloud services and open-source software. The shared firmware core and the single Web Bluetooth console make the project easy to reproduce and extend.

References

1. Seeed/YF-S201 Water Flow Sensor datasheet (F = 7.5·Q).
2. ElecFreaks, HC-SR04 user guide; JSN-SR04T waterproof module datasheet.
3. Adafruit IO MQTT API documentation.
4. Bureau of Indian Standards, IS 732: Code of practice for electrical wiring installations (safety reference).
5. Espressif Systems, ESP32 Series Datasheet and ESP32 Technical Reference Manual, espressif.com.
6. Espressif Systems, Arduino-ESP32 documentation, docs.espressif.com/projects/arduino-esp32.
7. H2zero, NimBLE-Arduino library documentation, github.com/h2zero/NimBLE-Arduino.
8. Bluetooth SIG, Bluetooth Core Specification 4.2 — Generic Attribute Profile (GATT).
9. W3C Web Bluetooth Community Group, Web Bluetooth specification, webbluetoothcg.github.io/web-bluetooth.
10. B. Blanchon, ArduinoJson 7 documentation, arduinojson.org.
11. Adafruit Industries, Adafruit IO documentation and MQTT API, io.adafruit.com/api/docs.

Appendix A: Project Source Code

The complete code is in the repository folder **P07_Water_Tank_Pump_Controller** (https://github.com/rohitkukreja705/esp32-iot-projects).

config.h

```
#pragma once
// ===== P07 Water Tank Level & Pump Controller — configuration =====
#define WIFI_SSID     ""
#define WIFI_PASS     ""
#define DEVICE_PIN    "1234"
#define AIO_USERNAME  ""
#define AIO_KEY       ""

// ----- pins -----
#define PIN_TRIG      25     // HC-SR04 TRIG (sensor mounted in the tank lid, facing down)
#define PIN_ECHO      34     // ECHO via 1k/2k divider
#define PIN_FLOW      27     // YF-S201 signal via 10k/20k divider
#define PIN_RELAY     26     // pump relay IN
#define RELAY_ACTIVE_LOW true
#define PIN_BUZZER    18

// ----- tank geometry defaults (set:depth=, set:fullcm=) -----
#define DEF_DEPTH_CM   100   // sensor to tank bottom
#define DEF_FULL_CM    15    // sensor to the "100 %" water line
#define DEF_LOW_PCT    25    // auto: pump ON below
#define DEF_HIGH_PCT   90    // auto: pump OFF at/above
#define DEF_MAXRUN_MIN 30    // safety: max continuous run
#define DRYRUN_S       30    // pump ON but no flow for this long -> stop
#define FLOW_PULSES_PER_LITRE 450.0f   // YF-S201: F(Hz) = 7.5 x L/min

#define SAMPLE_MS      1000
#define CLOUD_MS       15000
```

P07_Water_Tank_Pump_Controller.ino

```
/*
 * P07 — Water Tank Level Monitor & Smart Pump Controller
 * ======================================================
 * Peripherals: HC-SR04 ultrasonic (level), YF-S201 flow sensor,
 *              relay (pump), buzzer
 * Wi-Fi  : Adafruit IO — level %, flow, pump status, AUTO/MANUAL mode,
 *          remote pump switch from the dashboard
 * BLE    : live tank gauge, pump/mode control, tank calibration
 * Logic  : AUTO mode fills between LOW and HIGH thresholds with hysteresis;
 *          protections: dry-run (no flow), overflow, maximum run time;
 *          median filter on distance; litres pumped counter
 */
#include "config.h"
#include "IoTCore.h"
#include "Cloud.h"

cloud::AdafruitIO aio;
const char* SUB_FEEDS[] = { "p07-pump", "p07-mode" };

volatile uint32_t flowPulses = 0;
void IRAM_ATTR onFlowPulse() { flowPulses = flowPulses + 1; }

float distCm = NAN, levelPct = NAN, flowLpm = 0, litres = 0;
bool pumpOn = false, autoMode = true, alarmOn = false;
```

```
String fault = "";
uint32_t pumpStart = 0, noFlowSince = 0, lastSample = 0, lastCloud = 0, lastFlowCalc = 0;
float depthCm, fullCm, lowPct, highPct, maxRunMin;

void loadSettings() {
  depthCm   = iot::settingF("depth",  DEF_DEPTH_CM);
  fullCm    = iot::settingF("fullcm", DEF_FULL_CM);
  lowPct    = iot::settingF("low",    DEF_LOW_PCT);
  highPct   = iot::settingF("high",   DEF_HIGH_PCT);
  maxRunMin = iot::settingF("maxrun", DEF_MAXRUN_MIN);
}

float pingOnce() {
  digitalWrite(PIN_TRIG, LOW); delayMicroseconds(3);
  digitalWrite(PIN_TRIG, HIGH); delayMicroseconds(10); digitalWrite(PIN_TRIG, LOW);
  unsigned long us = pulseIn(PIN_ECHO, HIGH, 30000);
  return us ? us * 0.0343 / 2.0 : NAN;
}

// median of 5 pings rejects splashes and stray echoes
float readDistance() {
  float v[5]; int n = 0;
  for (int i = 0; i < 5; i++) { float d = pingOnce(); if (!isnan(d)) v[n++] = d; delay(15); }
  if (n == 0) return NAN;
  for (int i = 0; i < n; i++) for (int j = i + 1; j < n; j++) if (v[j] < v[i]) { float t = v[i]; v[i] =
v[j]; v[j] = t; }
  return v[n / 2];
}

void setPump(bool on, const String& why) {
  if (on == pumpOn) return;
  if (on && fault.length()) { iot::sendEvent("pump_blocked", "Clear fault first: " + fault); return; }
  pumpOn = on;
  digitalWrite(PIN_RELAY, (on ^ RELAY_ACTIVE_LOW) ? HIGH : LOW);
  if (on) { pumpStart = millis(); noFlowSince = millis(); }
  iot::sendEvent("pump", String(on ? "ON" : "OFF") + " - " + why);
  aio.publish("p07-pump", on ? "ON" : "OFF");
}

void raiseFault(const String& f) {
  fault = f; alarmOn = true;
  setPump(false, f);
  iot::sendEvent("fault", f);
}

void control() {
  uint32_t now = millis();
  // flow rate once per second
  if (now - lastFlowCalc >= 1000) {
    noInterrupts(); uint32_t p = flowPulses; flowPulses = 0; interrupts();
    float secs = (now - lastFlowCalc) / 1000.0;
    lastFlowCalc = now;
    flowLpm = (p / FLOW_PULSES_PER_LITRE) * 60.0 / secs;
    litres += p / FLOW_PULSES_PER_LITRE;
  }
  if (isnan(levelPct)) return;

  if (levelPct >= 98 && pumpOn) raiseFault("Overflow risk");
  if (pumpOn) {
    if (flowLpm > 0.5) noFlowSince = now;
    if (now - noFlowSince > DRYRUN_S * 1000UL) raiseFault("Dry run - no water flow");
    if (now - pumpStart > maxRunMin * 60000UL) raiseFault("Max run time reached");
  }
```

```
  if (autoMode && fault.isEmpty()) {
    if (!pumpOn && levelPct < lowPct)  setPump(true, "level low (auto)");
    if (pumpOn && levelPct >= highPct) setPump(false, "tank full (auto)");
  }
}

void sendBle() {
  JsonDocument d;
  d["lvl"] = iot::rnd(levelPct, 0); d["cm"] = iot::rnd(distCm, 1);
  d["flow"] = iot::rnd(flowLpm, 2); d["lit"] = iot::rnd(litres, 1);
  d["pump"] = pumpOn; d["mode"] = autoMode ? "AUTO" : "MANUAL";
  d["fault"] = fault; d["cloud"] = aio.connected();
  iot::sendData(d);
}

void onFeed(const String& feed, const String& value) {
  if (feed == "p07-pump") setPump(value.equalsIgnoreCase("ON"), "cloud");
  if (feed == "p07-mode") { autoMode = value.equalsIgnoreCase("AUTO"); iot::saveSetting("auto", autoMode
? "1" : "0"); }
}

// Commands: pump:on|off  mode:auto|manual  clear  resetvol
bool onCommand(const String& cmd, const String& arg, bool authed) {
  if (cmd == "setting") { loadSettings(); return true; }
  if (cmd == "pump" || cmd == "mode" || cmd == "clear" || cmd == "resetvol") {
    if (!authed) { iot::emitAck(cmd, false, "send auth:<PIN> first"); return true; }
    if (cmd == "pump") setPump(arg == "on", "BLE");
    if (cmd == "mode") {
      autoMode = arg == "auto"; iot::saveSetting("auto", autoMode ? "1" : "0");
      aio.publish("p07-mode", autoMode ? "AUTO" : "MANUAL");
    }
    if (cmd == "clear") { fault = ""; alarmOn = false; digitalWrite(PIN_BUZZER, LOW); }
    if (cmd == "resetvol") litres = 0;
    iot::emitAck(cmd, true); sendBle();
    return true;
  }
  return false;
}

void setup() {
  pinMode(PIN_TRIG, OUTPUT); pinMode(PIN_ECHO, INPUT);
  pinMode(PIN_RELAY, OUTPUT); pinMode(PIN_BUZZER, OUTPUT);
  pinMode(PIN_FLOW, INPUT);
  digitalWrite(PIN_RELAY, RELAY_ACTIVE_LOW ? HIGH : LOW);      // pump off at boot
  attachInterrupt(digitalPinToInterrupt(PIN_FLOW), onFlowPulse, FALLING);
  iot::begin("P07", onCommand);
  loadSettings();
  autoMode = iot::setting("auto", "1") == "1";
  // false: never switch the pump on from a stale cloud value after a reboot
  aio.begin(onFeed, SUB_FEEDS, 2, false);
  lastFlowCalc = millis();
}

void loop() {
  iot::loop();
  aio.loop();
  control();
  uint32_t now = millis();
  if (alarmOn) digitalWrite(PIN_BUZZER, (now / 300) % 2);
  if (now - lastSample >= SAMPLE_MS) {
    lastSample = now;
    distCm = readDistance();
    if (!isnan(distCm))
```

```
      levelPct = constrain((depthCm - distCm) / (depthCm - fullCm) * 100.0, 0, 100);
    sendBle();
  }
  if (aio.connected() && now - lastCloud >= CLOUD_MS && !isnan(levelPct)) {
    lastCloud = now;
    aio.publish("p07-level", levelPct, 0);
    aio.publish("p07-flow", flowLpm, 2);
  }
}
```

Appendix B: Shared Library Code

These two headers are identical in every project folder (master copies in /common).

IoTCore.h

```
/*
 * IoTCore.h  —  shared runtime for all ESP32 IoT projects in this repo
 * ---------------------------------------------------------------------
 *  - Wi-Fi station with automatic reconnect (credentials stored in flash)
 *  - BLE server using the Nordic UART Service (NUS) layout, so the device
 *    works with the Web Bluetooth dashboard in /docs AND with generic apps
 *    such as "nRF Connect" or "Serial Bluetooth Terminal"
 *  - Line-based command protocol shared by BLE and USB Serial
 *  - Persistent settings (Preferences / NVS): Wi-Fi, cloud keys, PIN, thresholds
 *  - NTP time sync (IST) for timestamps
 *
 *  Wire protocol
 *  -------------
 *  Phone -> ESP32 (write to RX characteristic, text):
 *      auth:1234              unlock control + settings for this connection
 *      status                 device replies with a status JSON line
 *      set:key=value          store a setting (Wi-Fi, cloud key, threshold…)
 *      get:key                read back a setting (secrets are masked)
 *      wifi:SSID,PASSWORD     shortcut for set:ssid + set:pass + reconnect
 *      reboot                 restart the board
 *      <anything else>        forwarded to the project's own command handler
 *  ESP32 -> Phone (notify on TX characteristic):
 *      one JSON object per line, terminated by '\n'
 *      {"type":"data",...}  {"type":"status",...}  {"type":"ack",...}
 *      {"type":"event",...} {"type":"err",...}
 *  Lines longer than one BLE packet are split into chunks; the receiver
 *  joins chunks until it sees '\n'.
 *
 *  Library: NimBLE-Arduino 2.x (lighter than Bluedroid, leaves more RAM for TLS)
 *  Board:   ESP32 Dev Module, Partition Scheme "Minimal SPIFFS (1.9MB APP with OTA)"
 */
#pragma once

#include <Arduino.h>
#include <WiFi.h>
#include <Preferences.h>
#include <NimBLEDevice.h>
#include <ArduinoJson.h>
#include <time.h>

#ifndef DEVICE_PIN
#define DEVICE_PIN "1234"
#endif
#ifndef WIFI_SSID
#define WIFI_SSID ""
#endif
#ifndef WIFI_PASS
#define WIFI_PASS ""
#endif

namespace iot {

// Nordic UART Service UUIDs (widely supported by BLE terminal apps)
static const char* NUS_SERVICE = "6E400001-B5A3-F393-E0A9-E50E24DCCA9E";
static const char* NUS_RX      = "6E400002-B5A3-F393-E0A9-E50E24DCCA9E";  // phone writes here
static const char* NUS_TX      = "6E400003-B5A3-F393-E0A9-E50E24DCCA9E";  // ESP32 notifies here
```

```
// Project command handler: return true if the command was recognised.
using CommandHandler = bool (*)(const String& cmd, const String& arg, bool authed);

// ---------------------------------------------------------------- state
struct CmdMsg { char text[256]; bool fromBle; };

static Preferences        prefs;
static NimBLEServer*      bleServer   = nullptr;
static NimBLECharacteristic* txChr    = nullptr;
static QueueHandle_t      cmdQueue    = nullptr;
static CommandHandler     userHandler = nullptr;
static String             deviceName;
static String             projectId;
static volatile uint16_t  bleMtu      = 23;
static volatile bool      bleAuthed   = false;   // reset on every new connection
static bool               serialAuthed = true;   // USB = physical access, trusted
static bool               timeSynced  = false;
static uint32_t           lastWifiTry = 0;
static String             rxLine;                // assembles multi-packet writes

// ---------------------------------------------------------------- settings
inline String setting(const char* key, const char* def = "") {
  return prefs.isKey(key) ? prefs.getString(key, def) : String(def);
}
inline void saveSetting(const char* key, const String& v) { prefs.putString(key, v); }
inline float settingF(const char* key, float def) {
  String s = setting(key, "");
  return s.length() ? s.toFloat() : def;
}

// ---------------------------------------------------------------- BLE
inline bool bleConnected() { return bleServer && bleServer->getConnectedCount() > 0; }

// Send one line (adds '\n'), split to fit the negotiated MTU.
inline void bleSend(const String& line) {
  if (!bleConnected() || !txChr) return;
  String msg = line + "\n";
  size_t chunk = (bleMtu > 23 ? bleMtu : 23) - 3;
  for (size_t i = 0; i < msg.length(); i += chunk) {
    size_t n = min(chunk, msg.length() - i);
    txChr->setValue((const uint8_t*)msg.c_str() + i, n);
    txChr->notify();
    if (msg.length() > chunk) delay(8);   // give the stack time between chunks
  }
}

// Send to BLE and mirror to USB Serial.
inline void emit(const String& line) {
  Serial.println(line);
  bleSend(line);
}

inline void emitAck(const String& cmd, bool ok, const String& msg = "") {
  JsonDocument d;
  d["type"] = ok ? "ack" : "err";
  d["cmd"] = cmd;
  if (msg.length()) d["msg"] = msg;
  String s; serializeJson(d, s); emit(s);
}

inline void enqueue(const String& s, bool fromBle) {
  if (!cmdQueue || s.length() == 0) return;
```

```
  CmdMsg m; memset(&m, 0, sizeof(m));
  strncpy(m.text, s.c_str(), sizeof(m.text) - 1);
  m.fromBle = fromBle;
  xQueueSend(cmdQueue, &m, 0);
}

class ServerCB : public NimBLEServerCallbacks {
  void onConnect(NimBLEServer* s, NimBLEConnInfo& info) override {
    bleAuthed = false;
    rxLine = "";
    Serial.printf("[BLE] connected %s\n", info.getAddress().toString().c_str());
  }
  void onDisconnect(NimBLEServer* s, NimBLEConnInfo& info, int reason) override {
    bleAuthed = false;
    bleMtu = 23;
    Serial.printf("[BLE] disconnected (reason %d), advertising again\n", reason);
    // NimBLE restarts advertising automatically (advertiseOnDisconnect = true)
  }
  void onMTUChange(uint16_t mtu, NimBLEConnInfo& info) override {
    bleMtu = mtu;
    Serial.printf("[BLE] MTU %u\n", mtu);
  }
};

class RxCB : public NimBLECharacteristicCallbacks {
  void onWrite(NimBLECharacteristic* c, NimBLEConnInfo& info) override {
    std::string v = c->getValue();
    // Accept either a single write without newline, or several writes
    // that together form lines terminated by '\n'.
    for (char ch : v) {
      if (ch == '\r') continue;
      if (ch == '\n') { enqueue(rxLine, true); rxLine = ""; }
      else if (rxLine.length() < 250) rxLine += ch;
    }
    if (v.find('\n') == std::string::npos && rxLine.length() && rxLine.length() == v.length()) {
      enqueue(rxLine, true);  // plain one-shot write (e.g. typed in nRF Connect)
      rxLine = "";
    }
  }
};

inline void bleBegin() {
  uint8_t mac[6]; WiFi.macAddress(mac);
  char suffix[6]; snprintf(suffix, sizeof(suffix), "%02X%02X", mac[4], mac[5]);
  deviceName = "IOT-" + projectId + "-" + suffix;

  NimBLEDevice::init(deviceName.c_str());
  NimBLEDevice::setMTU(247);
  bleServer = NimBLEDevice::createServer();
  bleServer->setCallbacks(new ServerCB());

  NimBLEService* svc = bleServer->createService(NUS_SERVICE);
  txChr = svc->createCharacteristic(NUS_TX, NIMBLE_PROPERTY::NOTIFY | NIMBLE_PROPERTY::READ);
  NimBLECharacteristic* rx = svc->createCharacteristic(
      NUS_RX, NIMBLE_PROPERTY::WRITE | NIMBLE_PROPERTY::WRITE_NR, 512);
  rx->setCallbacks(new RxCB());
  // (NimBLE 2.x starts services automatically when advertising begins)

  // 128-bit UUID goes in the advertisement, the name in the scan response
  // (both together would not fit in one 31-byte packet).
  NimBLEAdvertisementData adv, scan;
  adv.setFlags(BLE_HS_ADV_F_DISC_GEN | BLE_HS_ADV_F_BREDR_UNSUP);
  adv.setCompleteServices(NimBLEUUID(NUS_SERVICE));
```

```
  scan.setName(deviceName.c_str());
  NimBLEAdvertising* a = NimBLEDevice::getAdvertising();
  a->setAdvertisementData(adv);
  a->setScanResponseData(scan);
  a->start();
  Serial.printf("[BLE] advertising as %s\n", deviceName.c_str());
}

// ---------------------------------------------------------------- Wi-Fi
inline bool wifiOK() { return WiFi.status() == WL_CONNECTED; }

inline void wifiConnect() {
  String ssid = setting("ssid", WIFI_SSID);
  String pass = setting("pass", WIFI_PASS);
  if (ssid.isEmpty()) {
    Serial.println("[WiFi] no SSID yet - send  wifi:SSID,PASSWORD  over BLE");
    return;
  }
  Serial.printf("[WiFi] connecting to %s\n", ssid.c_str());
  WiFi.disconnect();
  WiFi.begin(ssid.c_str(), pass.c_str());
  lastWifiTry = millis();
}

inline void wifiLoop() {
  static bool wasUp = false;
  bool up = wifiOK();
  if (up && !wasUp) {
    Serial.printf("[WiFi] connected, IP %s, RSSI %d\n",
                  WiFi.localIP().toString().c_str(), WiFi.RSSI());
    configTzTime("IST-5:30", "pool.ntp.org", "time.google.com");
  }
  if (!up && millis() - lastWifiTry > 20000) wifiConnect();   // retry every 20 s
  wasUp = up;
  if (up && !timeSynced && time(nullptr) > 1700000000) {
    timeSynced = true;
    Serial.println("[NTP] time synced");
  }
}

// ISO-8601 local time, or uptime if NTP has not synced yet.
inline String timestamp() {
  time_t now = time(nullptr);
  if (now < 1700000000) return "uptime+" + String(millis() / 1000) + "s";
  struct tm t; localtime_r(&now, &t);
  char buf[32]; strftime(buf, sizeof(buf), "%Y-%m-%dT%H:%M:%S+05:30", &t);
  return String(buf);
}

// ------------------------------------------------------- built-in commands
inline bool isSecretKey(const String& k) {
  return k == "pass" || k.endsWith("key") || k == "pin";
}

inline void sendStatus() {
  JsonDocument d;
  d["type"] = "status";
  d["dev"] = deviceName;
  d["wifi"] = wifiOK();
  d["ssid"] = setting("ssid", WIFI_SSID);
  d["rssi"] = wifiOK() ? WiFi.RSSI() : 0;
  d["ip"] = wifiOK() ? WiFi.localIP().toString() : "";
  d["auth"] = bleAuthed;
```

```
  d["heap"] = ESP.getFreeHeap();
  d["up"] = millis() / 1000;
  d["time"] = timestamp();
  String s; serializeJson(d, s); emit(s);
}

inline void handleLine(const String& raw, bool fromBle) {
  String line = raw; line.trim();
  if (line.isEmpty()) return;
  int c = line.indexOf(':');
  String cmd = (c < 0 ? line : line.substring(0, c));
  String arg = (c < 0 ? "" : line.substring(c + 1));
  cmd.trim(); cmd.toLowerCase();
  bool authed = fromBle ? bleAuthed : serialAuthed;

  if (cmd == "auth") {
    bool ok = arg == setting("pin", DEVICE_PIN);
    if (fromBle) bleAuthed = ok;
    emitAck(cmd, ok, ok ? "unlocked" : "wrong PIN");
    return;
  }
  if (cmd == "status") { sendStatus(); return; }
  if (cmd == "ping")   { emitAck(cmd, true, "pong"); return; }

  // Everything below changes the device, so it needs the PIN over BLE.
  if (cmd == "set" || cmd == "wifi" || cmd == "reboot" || cmd == "get" || cmd == "forget") {
    if (!authed) { emitAck(cmd, false, "send auth:<PIN> first"); return; }
    if (cmd == "set") {
      int e = arg.indexOf('=');
      if (e <= 0) { emitAck(cmd, false, "use set:key=value"); return; }
      String k = arg.substring(0, e); k.trim();
      String v = arg.substring(e + 1); v.trim();
      if (k.length() > 15) { emitAck(cmd, false, "key too long"); return; }
      saveSetting(k.c_str(), v);
      emitAck(cmd, true, k + " saved");
      if (k == "ssid" || k == "pass") wifiConnect();
      // let the project react to changed thresholds etc.
      if (userHandler) userHandler("setting", k, true);
      return;
    }
    if (cmd == "get") {
      String v = setting(arg.c_str(), "");
      if (isSecretKey(arg) && v.length()) v = "****" + v.substring(max(0, (int)v.length() - 3));
      emitAck(cmd, true, arg + "=" + v);
      return;
    }
    if (cmd == "wifi") {
      int k = arg.indexOf(',');
      String ssid = k < 0 ? arg : arg.substring(0, k);
      String pass = k < 0 ? "" : arg.substring(k + 1);
      saveSetting("ssid", ssid); saveSetting("pass", pass);
      emitAck(cmd, true, "connecting to " + ssid);
      wifiConnect();
      return;
    }
    if (cmd == "forget") { prefs.clear(); emitAck(cmd, true, "all settings erased"); return; }
    if (cmd == "reboot") { emitAck(cmd, true, "rebooting"); delay(300); ESP.restart(); }
  }

  if (userHandler && userHandler(cmd, arg, authed)) return;
  emitAck(cmd, false, "unknown command");
}
```

```
// ------------------------------------------------------------ entry points
// Call once from setup().  id = short project id, e.g. "P01".
inline void begin(const char* id, CommandHandler handler) {
  Serial.begin(115200);
  delay(200);
  projectId = id;
  userHandler = handler;
  prefs.begin("iot", false);
  cmdQueue = xQueueCreate(8, sizeof(CmdMsg));
  WiFi.mode(WIFI_STA);
  WiFi.setAutoReconnect(true);
  // NOTE: Wi-Fi modem sleep must stay ON while BLE is running (ESP32 coexistence).
  bleBegin();
  wifiConnect();
  Serial.println("[CORE] ready - type 'status' here or connect over BLE");
}

// Call every loop() iteration.
inline void loop() {
  wifiLoop();
  // USB serial commands (handy while developing)
  static String serialLine;
  while (Serial.available()) {
    char ch = Serial.read();
    if (ch == '\n' || ch == '\r') { if (serialLine.length()) enqueue(serialLine, false); serialLine =
""; }
    else if (serialLine.length() < 250) serialLine += ch;
  }
  CmdMsg m;
  while (cmdQueue && xQueueReceive(cmdQueue, &m, 0) == pdTRUE) handleLine(String(m.text), m.fromBle);
}

// Round for compact JSON. NaN stays NaN, which ArduinoJson writes as null.
inline float rnd(float v, int dp = 1) {
  if (isnan(v)) return v;
  float m = powf(10, dp);
  return roundf(v * m) / m;
}

// Helper: build and send a {"type":"data",...} line from a filled document.
inline void sendData(JsonDocument& d) {
  d["type"] = "data";
  String s; serializeJson(d, s); emit(s);
}

inline void sendEvent(const String& name, const String& msg) {
  JsonDocument d;
  d["type"] = "event"; d["name"] = name; d["msg"] = msg; d["time"] = timestamp();
  String s; serializeJson(d, s); emit(s);
}

}  // namespace iot
```

Cloud.h

```
/*
 * Cloud.h  –  free-tier cloud helpers used by the projects
 * ---------------------------------------------------------
 *  ThingSpeak   : HTTPS REST "update" (free tier: 1 update / 15 s per channel)
 *  Adafruit IO  : MQTT over TLS (free tier: 30 data points / minute, 10 feeds)
 *  Supabase     : PostgREST over HTTPS (free tier: 500 MB Postgres)
 *
```

```
 *  Keys are read from settings (stored over BLE with set:key=value) so no
 *  secret has to be hard-coded or committed to GitHub:
 *     tskey    ThingSpeak Write API Key
 *     aiouser  Adafruit IO username        aiokey  Adafruit IO key
 *     sburl    https://<ref>.supabase.co   sbkey   Supabase publishable/anon key
 *
 *  TLS note: certificate checking is disabled with setInsecure() to keep the
 *  projects beginner-friendly and independent of CA rotations. Traffic is
 *  still encrypted. For production, pin the root CA with setCACert().
 */
#pragma once

#include <WiFiClientSecure.h>
#include <HTTPClient.h>
#include <PubSubClient.h>
#include "IoTCore.h"

// Optional compile-time defaults (normally left empty and set over BLE)
#ifndef TS_WRITE_KEY
#define TS_WRITE_KEY ""
#endif
#ifndef AIO_USERNAME
#define AIO_USERNAME ""
#endif
#ifndef AIO_KEY
#define AIO_KEY ""
#endif
#ifndef SUPABASE_URL
#define SUPABASE_URL ""
#endif
#ifndef SUPABASE_KEY
#define SUPABASE_KEY ""
#endif

namespace cloud {

// ============================================================== ThingSpeak
// vals[i] -> field(i+1). Pass NAN to skip a field. Returns true on success.
inline bool thingSpeakUpdate(const float* vals, int n, const String& status = "") {
  if (!iot::wifiOK()) return false;
  String key = iot::setting("tskey", TS_WRITE_KEY);
  if (key.isEmpty()) { Serial.println("[TS] no tskey set"); return false; }
  String url = "https://api.thingspeak.com/update?api_key=" + key;
  for (int i = 0; i < n && i < 8; i++)
    if (!isnan(vals[i])) url += "&field" + String(i + 1) + "=" + String(vals[i], 2);
  if (status.length()) {
    String st = status; st.replace(" ", "%20");
    url += "&status=" + st;
  }
  WiFiClientSecure client; client.setInsecure();
  HTTPClient http; http.setTimeout(8000);
  if (!http.begin(client, url)) return false;
  int code = http.GET();
  String body = http.getString();
  http.end();
  bool ok = code == 200 && body.toInt() > 0;   // ThingSpeak returns the entry id, 0 = rejected
  Serial.printf("[TS] HTTP %d entry=%s\n", code, body.c_str());
  return ok;
}

// ============================================================== Supabase
// Generic PostgREST call. method: GET/POST/PATCH. path e.g. "/rest/v1/readings"
inline int supabaseRequest(const char* method, const String& path, const String& body,
```

```
                          String* response = nullptr, const char* prefer = "return=minimal") {
  if (!iot::wifiOK()) return -1;
  String base = iot::setting("sburl", SUPABASE_URL);
  String key  = iot::setting("sbkey", SUPABASE_KEY);
  if (base.isEmpty() || key.isEmpty()) { Serial.println("[SB] sburl/sbkey not set"); return -2; }
  if (base.endsWith("/")) base.remove(base.length() - 1);
  WiFiClientSecure client; client.setInsecure();
  HTTPClient http; http.setTimeout(8000);
  if (!http.begin(client, base + path)) return -3;
  http.addHeader("apikey", key);
  http.addHeader("Authorization", "Bearer " + key);
  http.addHeader("Content-Type", "application/json");
  if (prefer) http.addHeader("Prefer", prefer);
  int code = http.sendRequest(method, body);
  if (response) *response = http.getString();
  else if (code >= 300) Serial.printf("[SB] %s\n", http.getString().c_str());
  http.end();
  Serial.printf("[SB] %s %s -> %d\n", method, path.c_str(), code);
  return code;
}

inline bool supabaseInsert(const char* table, const String& json) {
  int c = supabaseRequest("POST", String("/rest/v1/") + table, json);
  return c >= 200 && c < 300;
}

// filter is a PostgREST filter, e.g. "id=eq.1"
inline bool supabaseUpdate(const char* table, const String& filter, const String& json) {
  int c = supabaseRequest("PATCH", String("/rest/v1/") + table + "?" + filter, json);
  return c >= 200 && c < 300;
}

// query e.g. "select=*&id=eq.1"  -> JSON array text in out
inline bool supabaseSelect(const char* table, const String& query, String& out) {
  int c = supabaseRequest("GET", String("/rest/v1/") + table + "?" + query, "", &out, nullptr);
  return c == 200;
}

// ============================================================ Adafruit IO
// MQTT wrapper with a simple rate limiter to respect the 30 msg/min limit.
using FeedCallback = void (*)(const String& feed, const String& value);

class AdafruitIO {
 public:
  // requestLast = true asks Adafruit IO for each feed's last value on connect
  // (useful for switches changed while offline; unsafe for things like pumps)
  void begin(FeedCallback cb, const char* const* subFeeds, int nSub, bool requestLast = true) {
    cb_ = cb; subs_ = subFeeds; nSub_ = nSub; requestLast_ = requestLast;
    tls_.setInsecure();
    mqtt_.setClient(tls_);
    mqtt_.setServer("io.adafruit.com", 8883);
    mqtt_.setBufferSize(512);
    mqtt_.setKeepAlive(60);
    mqtt_.setCallback([](char* topic, byte* payload, unsigned int len) {
      String t(topic), v;
      for (unsigned int i = 0; i < len; i++) v += (char)payload[i];
      int p = t.lastIndexOf('/');
      String feed = t.substring(p + 1);
      if (instance()->cb_) instance()->cb_(feed, v);
    });
    instance() = this;
  }
```

```
  void loop() {
    if (!iot::wifiOK()) return;
    if (!mqtt_.connected()) {
      if (millis() - lastTry_ < 10000) return;
      lastTry_ = millis();
      user_ = iot::setting("aiouser", AIO_USERNAME);
      String key = iot::setting("aiokey", AIO_KEY);
      if (user_.isEmpty() || key.isEmpty()) { Serial.println("[AIO] aiouser/aiokey not set"); return; }
      String id = "esp32-" + String((uint32_t)ESP.getEfuseMac(), HEX);
      Serial.println("[AIO] connecting...");
      if (mqtt_.connect(id.c_str(), user_.c_str(), key.c_str())) {
        Serial.println("[AIO] connected");
        for (int i = 0; i < nSub_; i++) {
          mqtt_.subscribe(topic(subs_[i]).c_str());
          if (requestLast_) mqtt_.publish((topic(subs_[i]) + "/get").c_str(), "");
        }
        // subscribe to throttle/error topics so we learn about rate limits
        mqtt_.subscribe((user_ + "/throttle").c_str());
      } else {
        Serial.printf("[AIO] failed, state %d\n", mqtt_.state());
      }
    }
    mqtt_.loop();
  }

  bool connected() { return mqtt_.connected(); }

  // Returns false if not connected or if the per-minute budget is used up.
  bool publish(const char* feed, const String& value) {
    if (!mqtt_.connected()) return false;
    uint32_t now = millis();
    if (now - windowStart_ > 60000) { windowStart_ = now; sent_ = 0; }
    if (sent_ >= 28) { Serial.println("[AIO] rate budget used, skipping"); return false; }
    sent_++;
    return mqtt_.publish(topic(feed).c_str(), value.c_str());
  }
  bool publish(const char* feed, float v, int dp = 2) { return publish(feed, String(v, dp)); }

 private:
  String topic(const char* feed) { return user_ + "/feeds/" + feed; }
  static AdafruitIO*& instance() { static AdafruitIO* p = nullptr; return p; }

  WiFiClientSecure tls_;
  PubSubClient mqtt_;
  FeedCallback cb_ = nullptr;
  const char* const* subs_ = nullptr;
  int nSub_ = 0;
  bool requestLast_ = true;
  String user_;
  uint32_t lastTry_ = 0, windowStart_ = 0;
  int sent_ = 0;
};

}  // namespace cloud
```